

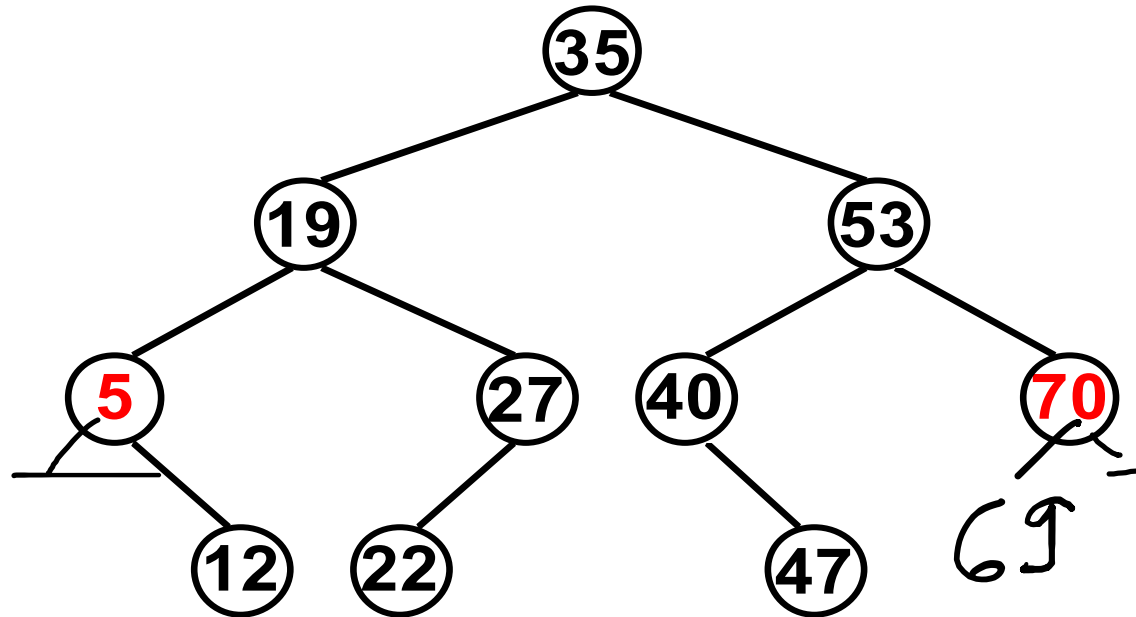
Arbori binari de cautare

Asigura cautarea rapida a nodului cu o anumita cheie.

Un arbore binar nevid este **arbore de cautare** daca indeplineste urmatoarele conditii:

- **cei doi subarbori** (daca exista) sunt, la randul lor, **arbori de cautare**
- **cheia nodului radacina** este **succesoarea** tuturor cheilor din subarborele stang si **predecesoarea** tuturor cheilor din subarborele drept
- $\text{orice info}(S) < \text{info}(R) < \text{orice info}(D)$
- S, D – arbori binari de cautare

Arbore de cautare cu chei intregi

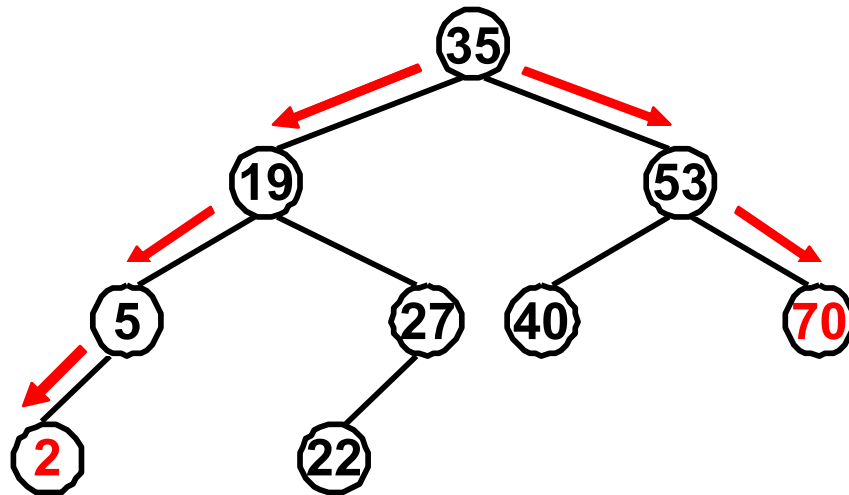


Parcurerea **S****R****D** $\Rightarrow$ ordine crescatoare

5, 12, 19, 22, 27, 35, 40, 47, 53, 70

Arbore de cautare cu chei intregi

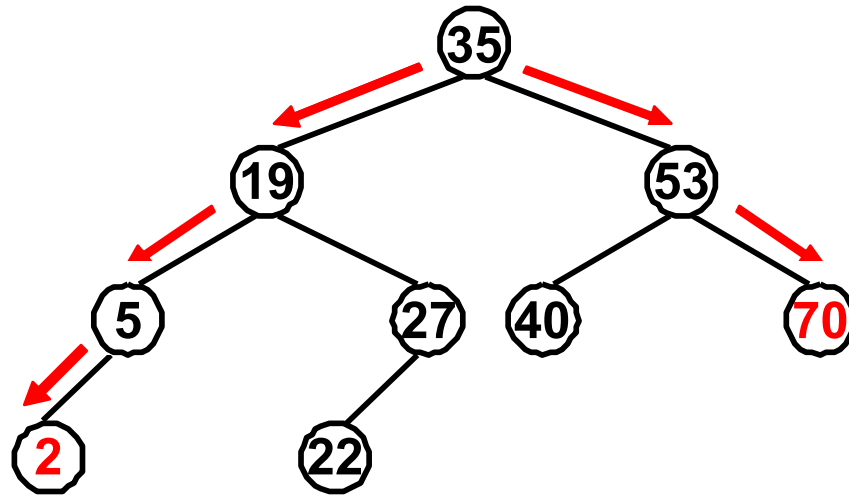
Valorile **minima** dintr-un arbore binar de cautare



```
TArb a;  
while (a->st != NULL)  
    a = a->st;  
//a - nodul cu informatia minima
```

Arbore de cautare cu chei intregi

Valorile **maxima** dintr-un arbore binar de cautare

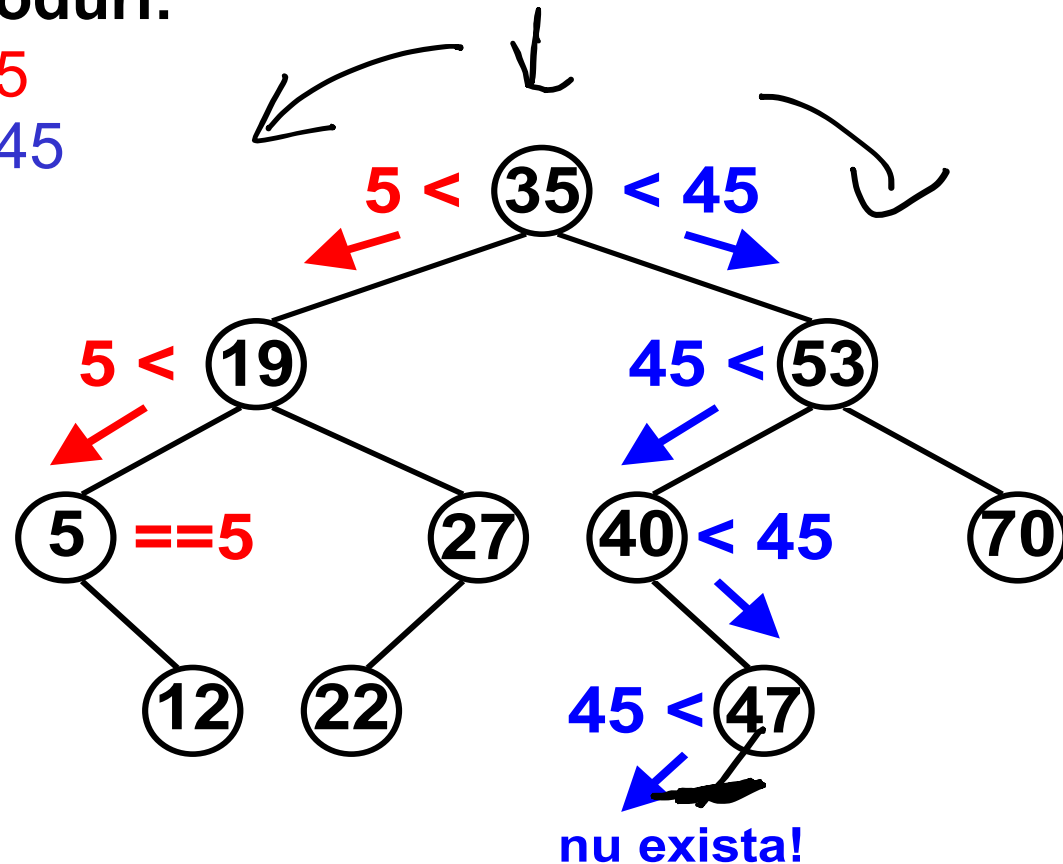


```
TArb a;  
while (a->dr != NULL)  
    a = a->dr;  
//a - nodul cu informatia maxima
```

Arbore de cautare cu chei intregi

Cautare noduri:

- cu cheia **5**
- cu cheia **45**



Numar maxim comparatii = numar niveluri

Arbore de cautare cu chei intregi

Cautarea nodului cu cheie data

```
TArb Cauta(TArb a, int cheie)
//NULL daca nu exista cheie in a
//nodul gasit
{ while (a) /* cat timp exista nod */
  {
    if (a->info == cheie) /* gasit */
      return a;
    if (cheie < a->info)
      a = a->st; /* deplasare stanga */
    else
      a = a->dr; /* deplasare dreapta */
  }
  return NULL;
}
```

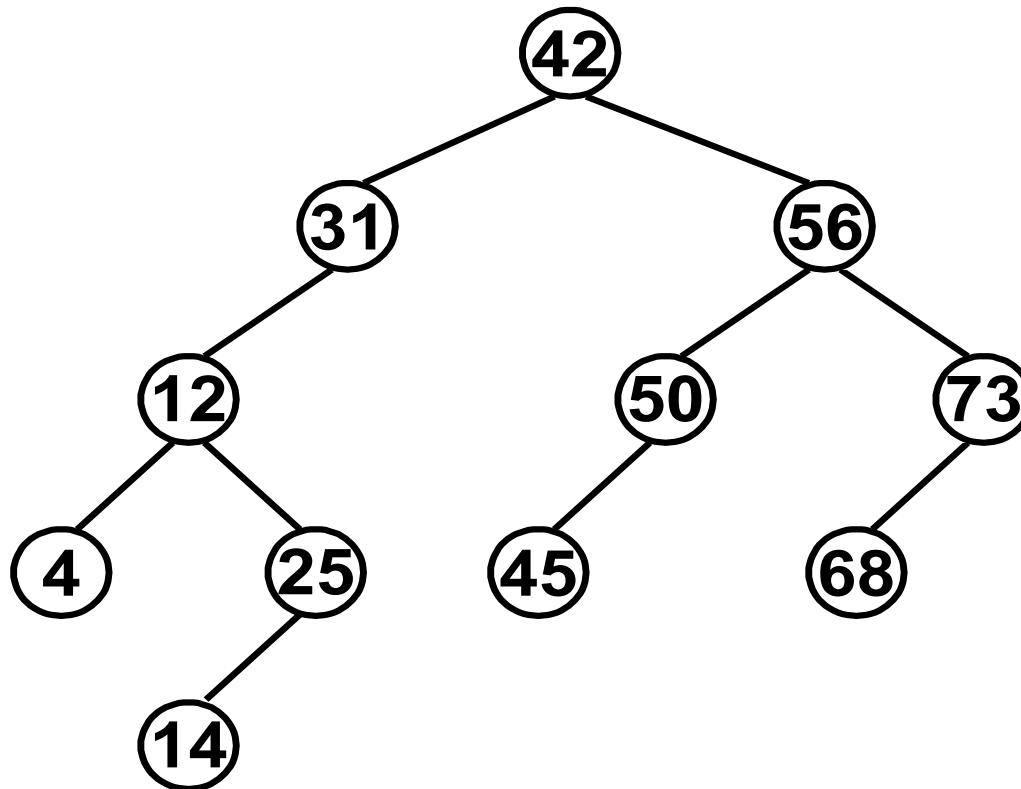
Arbore de cautare cu chei intregi

Cautarea nodului cu cheie data – cautare recursiva

```
TArb Cauta(TArb a, int cheie)  
{  
    if(!a) return 0;  
  
    if(a->info == x) return 1;  
  
    if(x < a->info) return Cauta(a->st, x);  
  
    return Cauta(a->dr, x);  
}
```

Construire arbore binar de cautare

Secventa chei: 42, 31, 56, 73, 12, 25, 50, 4, 45, 14, 68



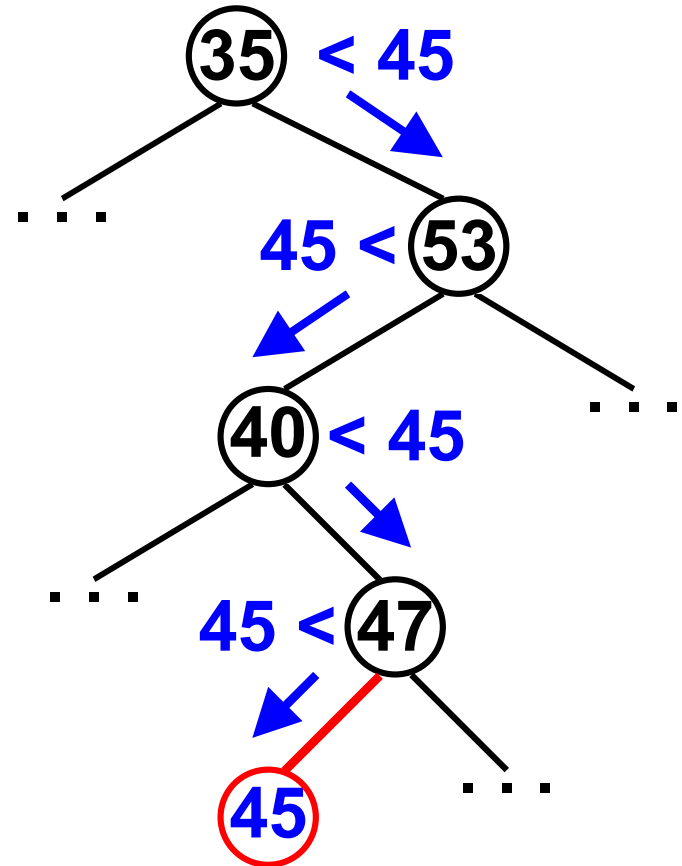
Inserare in arbore binar de cautare

Etape:

- cautarea locului de inserare
- inserarea propriu-zisa

Cazuri speciale:

- cheia de inserat
exista deja $\Rightarrow -1$
- nu exista spatiu
pentru un nou nod $\Rightarrow 0$



Inserare in arbore binar de cautare

```
int Insert_ABC(TArb* a, int x)
{
    TArb aux, n = *a, p = NULL;
    //n - parcurge arb, p - parintele nodului de ins
    int s; //fiu in care se face ins

    while (n)
    {
        if ( x == n->info) return -1;
        if ( x < n->info)
        {
            p = n; s = 0; n = n->st;
        }
        else
        {
            p = n; s = 1; n = n->dr;
        }
    }
}
```

Inserare in arbore binar de cautare

```
/* inserare */  
    aux = ConstrFrunza(x);  
    if(!aux) return 0; /* intoarce 0 */  
  
    if(!p) *a = aux;  
    else  
    {  
        if(s) p->dr = aux;  
        else p->st = aux;  
    }  
  
    return 1; /* intoarce 1 */  
}
```

Inserare recursiva in arbore binar

```
TArb Insert_ABC_R(TArb a, int x)  
{  
    if (a == NULL) return ConstrFrunza(x);  
  
    if (x < a->info)  
        a->st = Insert_ABC_R(a->st, x);  
    else  
        a->dr = Insert_ABC_R(a->dr, x);  
  
    return a;  
}
```

Inserare in arbore binar de cautare

```
int Insert_ABC(TArb *aa, int cheie)
{ /* cautare */
    while (*aa)
    { if ( cheie == (*aa)->info) return -1;
      if ( cheie < (*aa)->info)
          aa = &((*aa)->st);
      else
          aa = &((*aa)->dr);
    }

    /* inserare */
    *aa = ConstrFrunza(cheie);
    if(!*aa) return 0; /* intoarce 0 */
    return 1; /* intoarce 1 */
}
```

Constructie frunza

```
TArb ConstrFrunza(int x)
```

```
{
```

```
    TArb aux = (TArb)malloc(sizeof(TNod));
```

```
    if (!aux) return NULL;
```

```
    aux->info = x;
```

```
    aux->st = NULL;
```

```
    aux->dr = NULL;
```

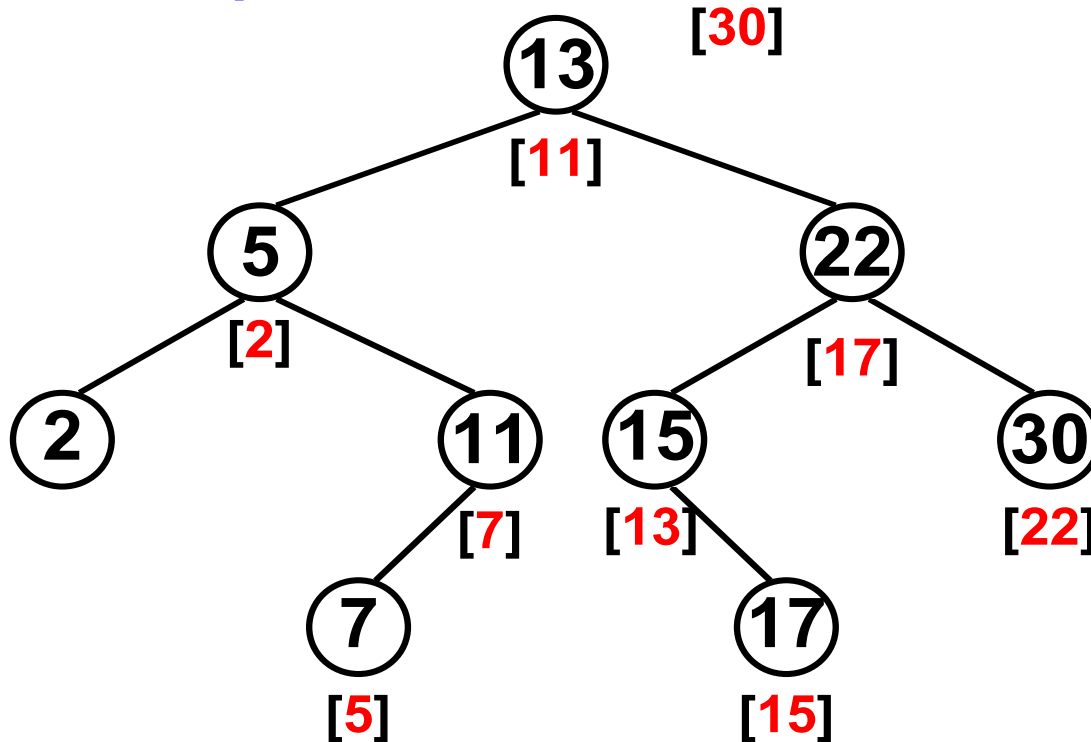
```
    return aux;
```

```
}
```

Test arbore binar de cautare

Bazata pe parcurgerea SRD:

- informatia din radacina (sub)arborelui analizat trebuie sa fie **> precedenta informatie analizata.**



Test arbore binar de cautare

Functia de test, apelata de utilizator cu un **singur parametru - arborele analizat**, initializeaza variabila care va memora ultima informatie analizata si apeleaza o functie auxiliara recursiva, cu doi parametri:

- **adresa (sub)arborelui analizat**
- **adresa variabilei de referinta**

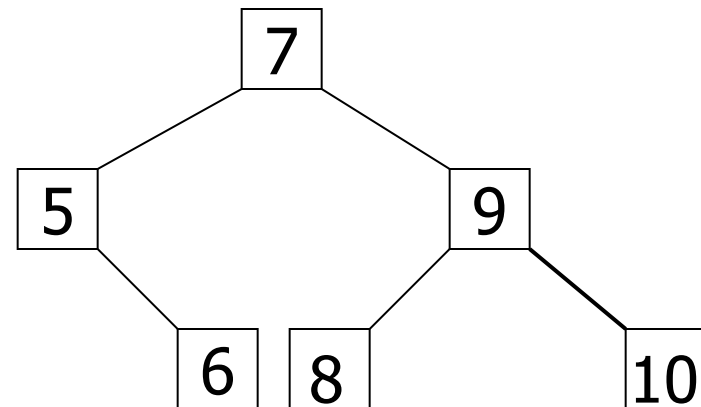
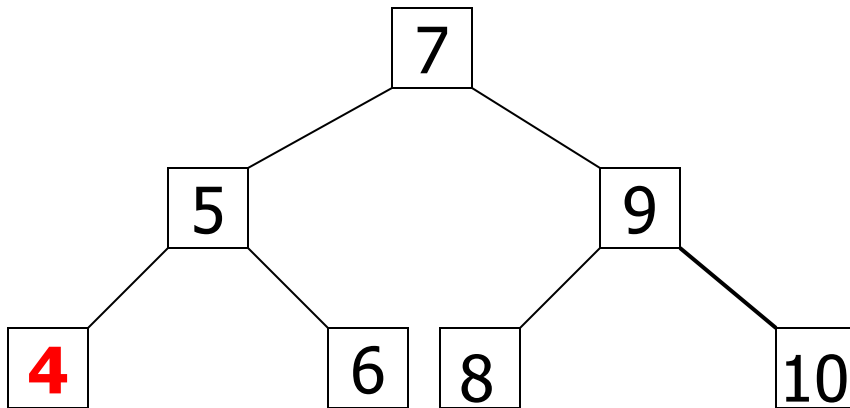
```
int TestAC(TArb a)
{ int ref;
  if (!a) return 1;
  ref = MinAC(a) - 1;
  return aux(a, &ref);
}
```


Test arbore binar de cautare

```
int aux(TArb a, int *p)  
{ /* test subarbore stang */  
    if (a->st)  
        if (!aux(a->st, p)) return 0;  
  
    /* test relatie de ordine */  
    if (a->info <= *p) return 0;  
  
    *p = a->info; /* actualizare referinta */  
  
    if (a->dr) /* test subarbore drept */  
        return aux(a->dr, p);  
    else  
        return 1;  
}
```

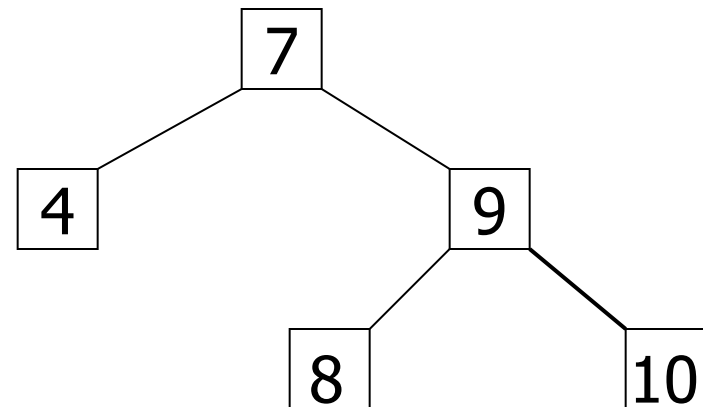
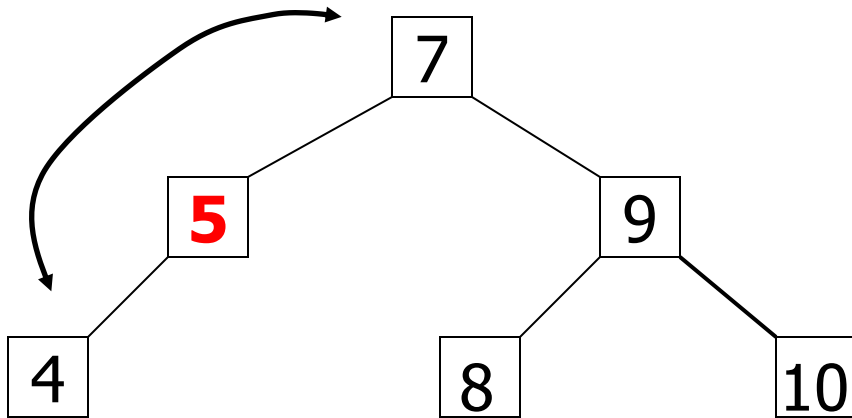
Eliminare nod

Eliminare frunza – eliminare nod cu info **4**



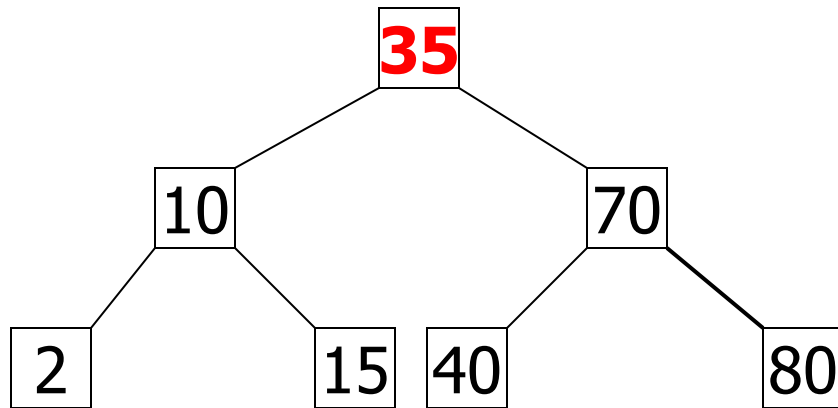
Eliminare nod

Eliminare nod de ordin 1 – eliminare nod cu info **5**

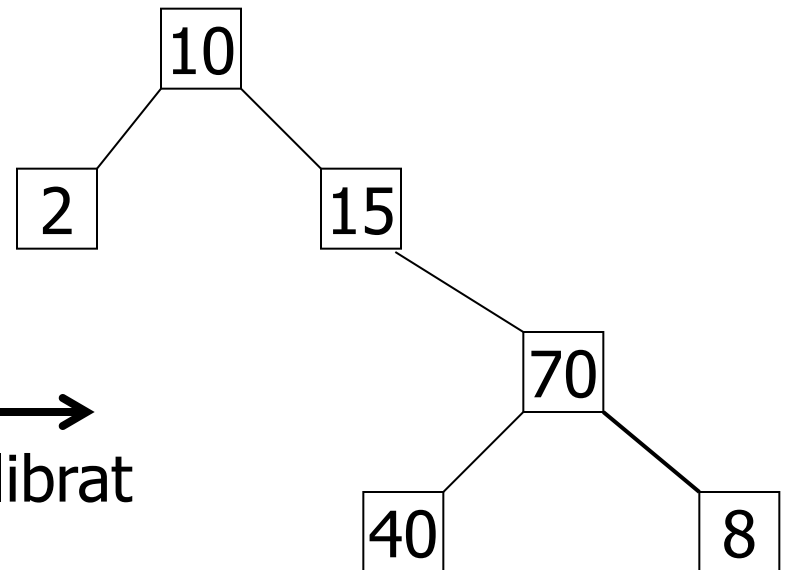


Eliminare nod

Eliminare nod de ordin 2: nodul cu info **35**

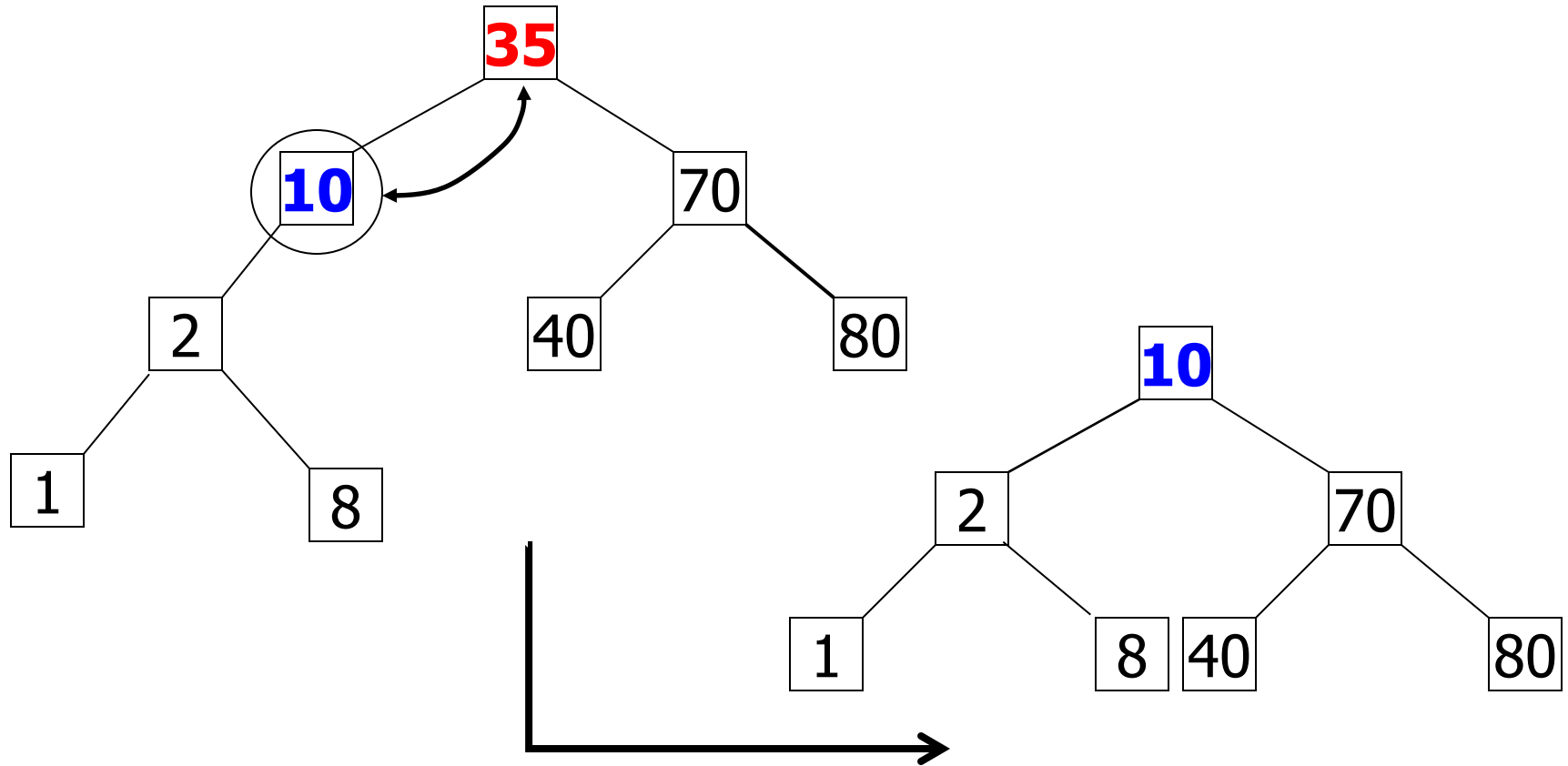


Arbore dezechilibrat



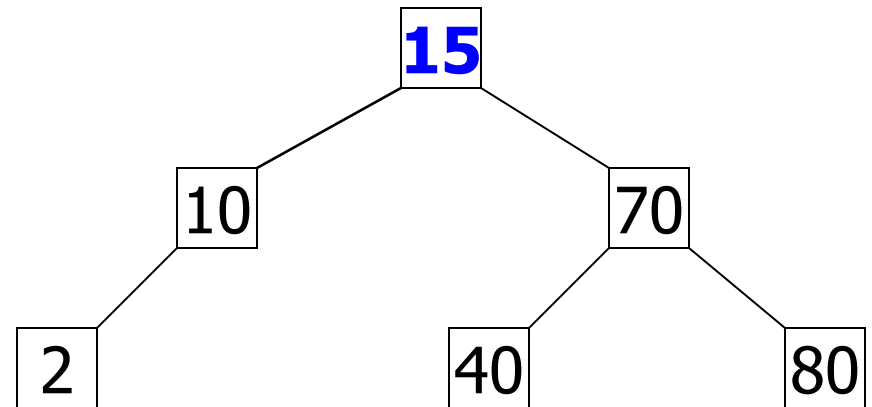
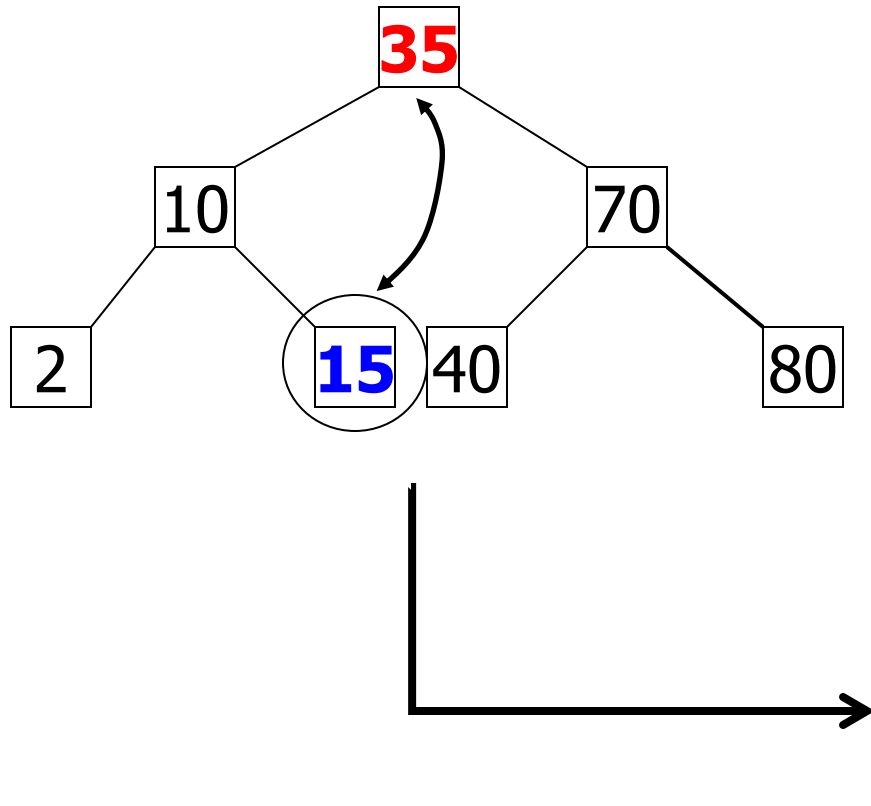
Eliminare nod

Eliminare nod de ordin 2: nodul cu info **35**



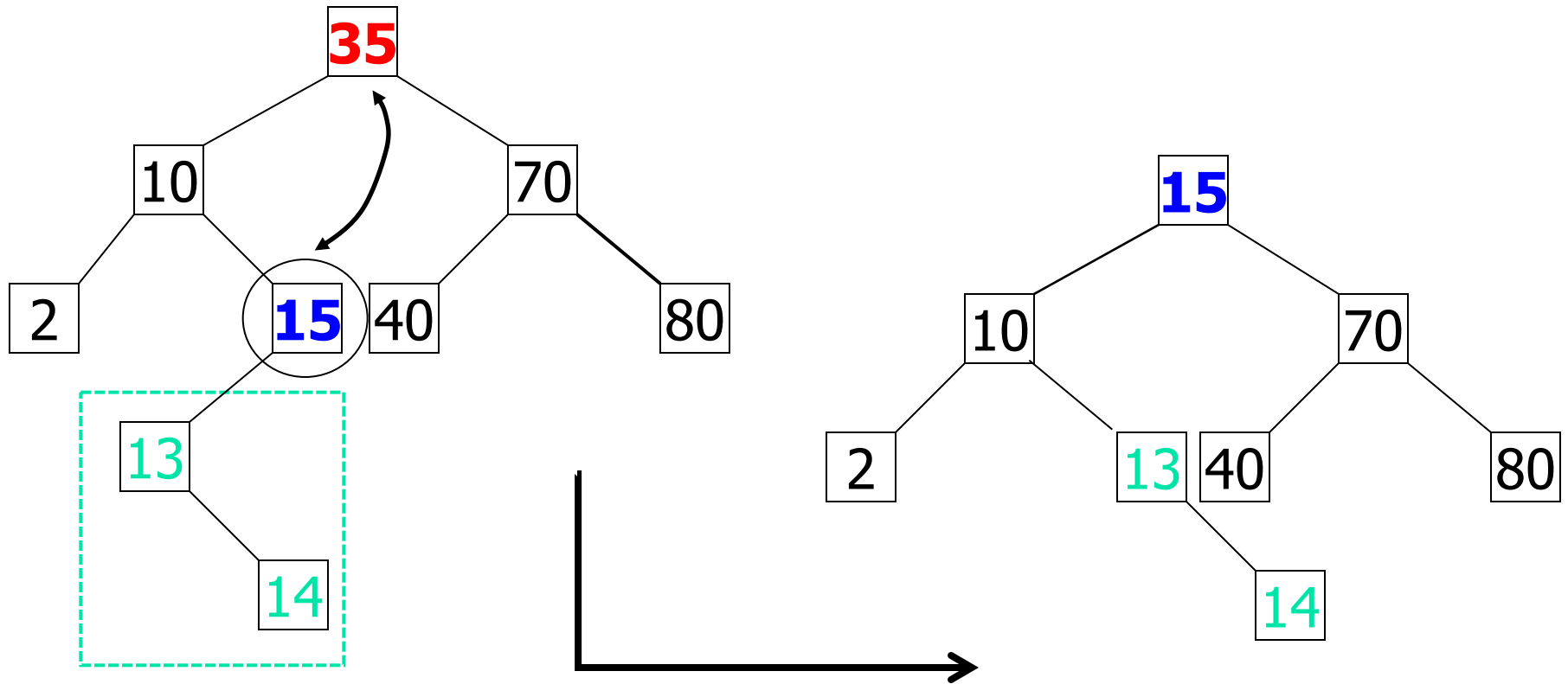
Eliminare nod

Eliminare nod de ordin 2 – eliminare nod cu info **35**



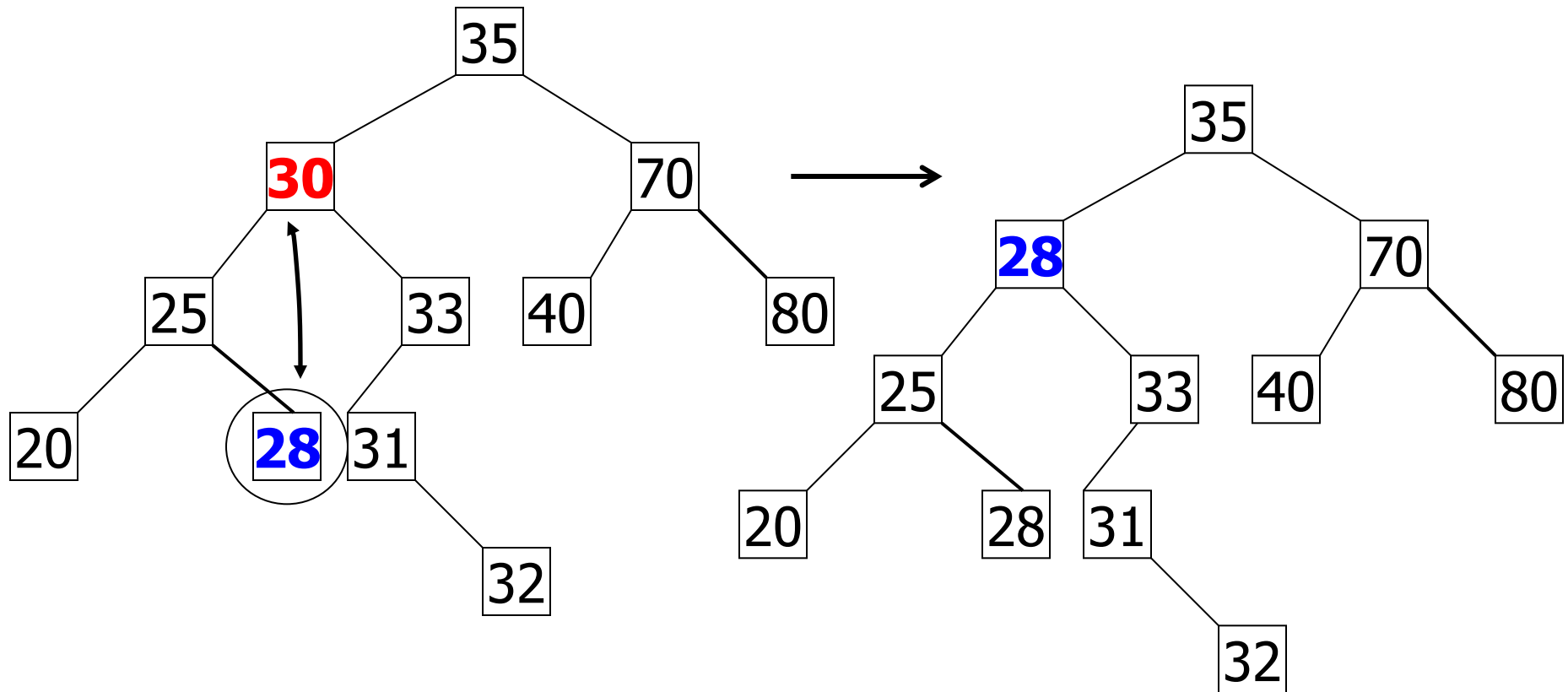
Eliminare nod

Eliminare nod de ordin 2 – eliminare nod cu info **35**



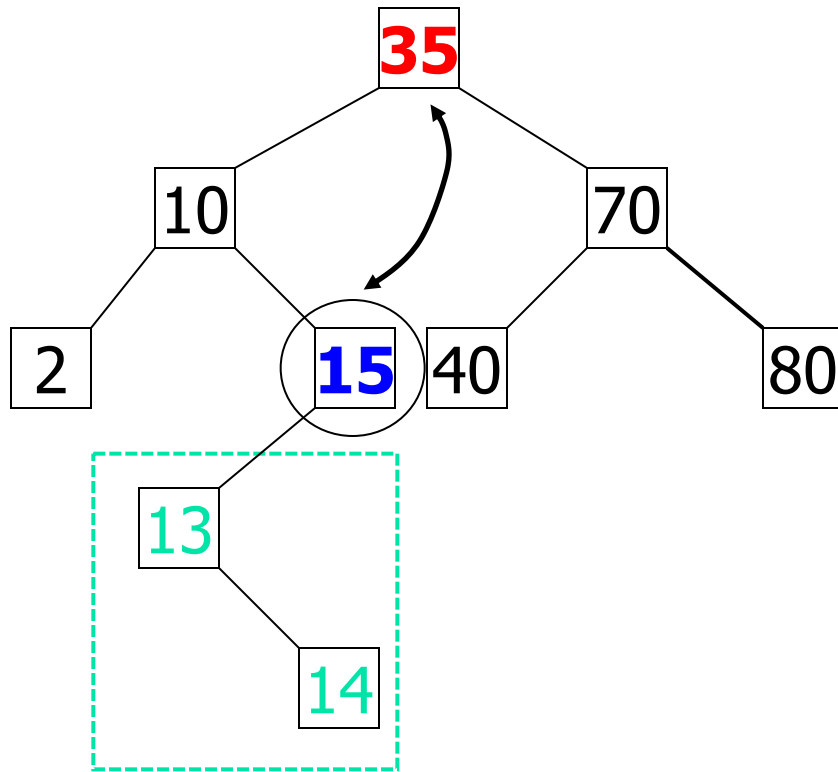
Eliminare nod

Eliminare nod de ordin 2 – eliminare nod cu info **30**



Eliminare nod

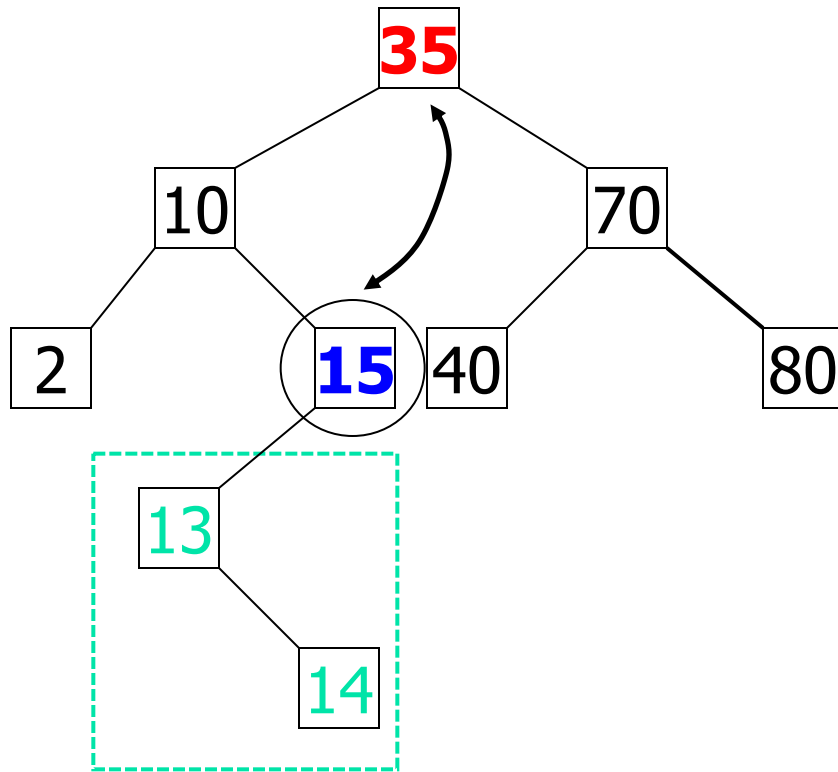
Eliminare nod de ordin 2 – eliminare nod cu info **35**



1. Localizare nod de elim (n) + parinte (p)
2. Daca $\text{ordin}(n) = 0$ sau 1
 1. Actualizeaza fiul parintelui p

Eliminare nod

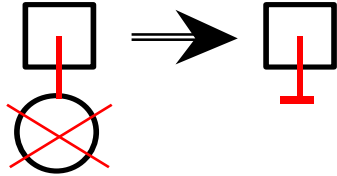
Eliminare nod de ordin 2 – eliminare nod cu info **35**



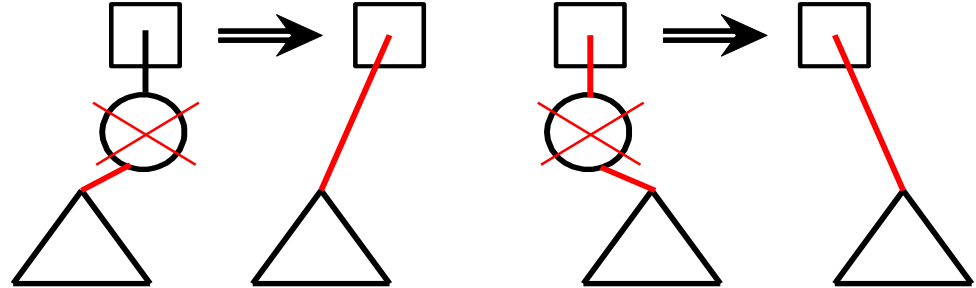
1. Localizare nod de elim (n) + parinte (p)
2. Daca $\text{ordin}(n) = 0$ sau 1
 1. Actualizeaza fiul parintelui p
3. Daca $\text{ordin}(n) = 2$
 1. Pivot = max(subarb stg pt n) + parinte pivot
 2. Plaseaza in n info din pivot
 3. Actualizeaza fiu parinte pivot

Eliminare nod

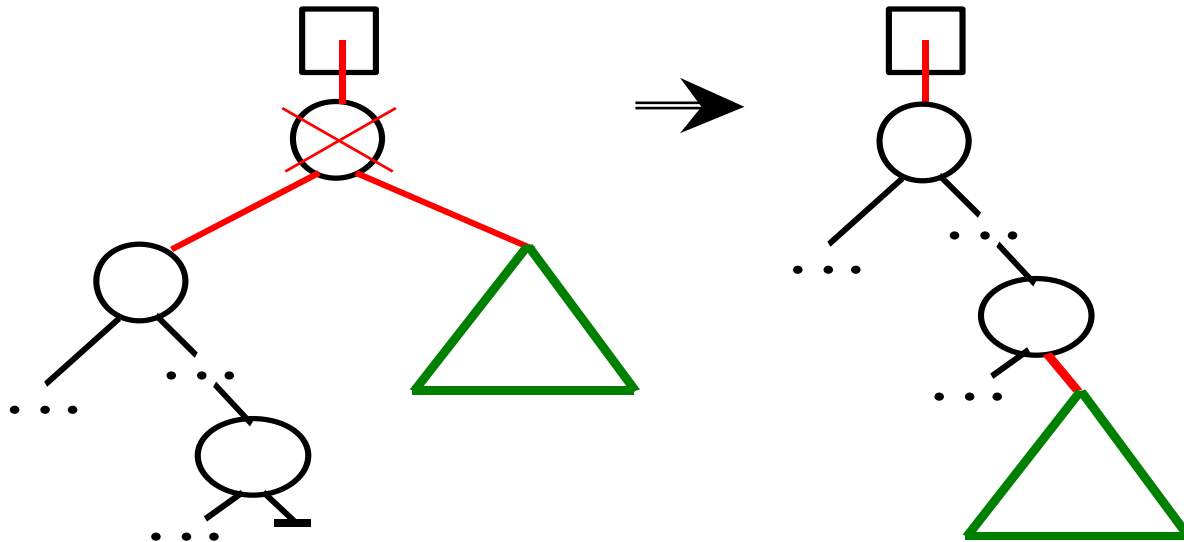
- frunza



- de ordin 1



- de ordin 2 – eliminare cu posibila dezechilibrare



Eliminare nod

- de ordin 2 – eliminare cu pivot stanga, caz particular

